DevSecOps Engineering & Application Security

Secure Authentication Workflow

Name:	T G JAI KOUSICK
Register No.:	192421367
Subject:	DevSecOps Engineering and its Application Security
Subject Code:	CSA5802
Department :	B.Tech (Information Technology)

1. Objective

This assessment designs a secure end-to-end authentication system covering user registration, login, and password recovery. It applies shift-left security by embedding controls at each stage of the software development lifecycle (SDLC) rather than only at deployment, maps each workflow against the OWASP Top 10 and the STRIDE threat-modeling framework, and presents supporting architecture and flow diagrams. The work corresponds to Bloom's Apply level (L3): known security principles — hashing, token management, rate limiting, defence in depth — are applied to a concrete system design.

2. System Architecture

The system follows a layered architecture: a client application communicates over TLS 1.3 with a WAF/API gateway that performs rate limiting and bot detection before requests reach the authentication microservice. The microservice is the single point of trust for identity operations and delegates to supporting services — an MFA/OTP service, a secrets vault for cryptographic key material, an email/SMS gateway for out-of-band communication, and a SIEM for centralised audit logging. Persistent data is split across a user store (password hashes only, never plaintext), a short-lived session/token store, and a dedicated password-reset token store.

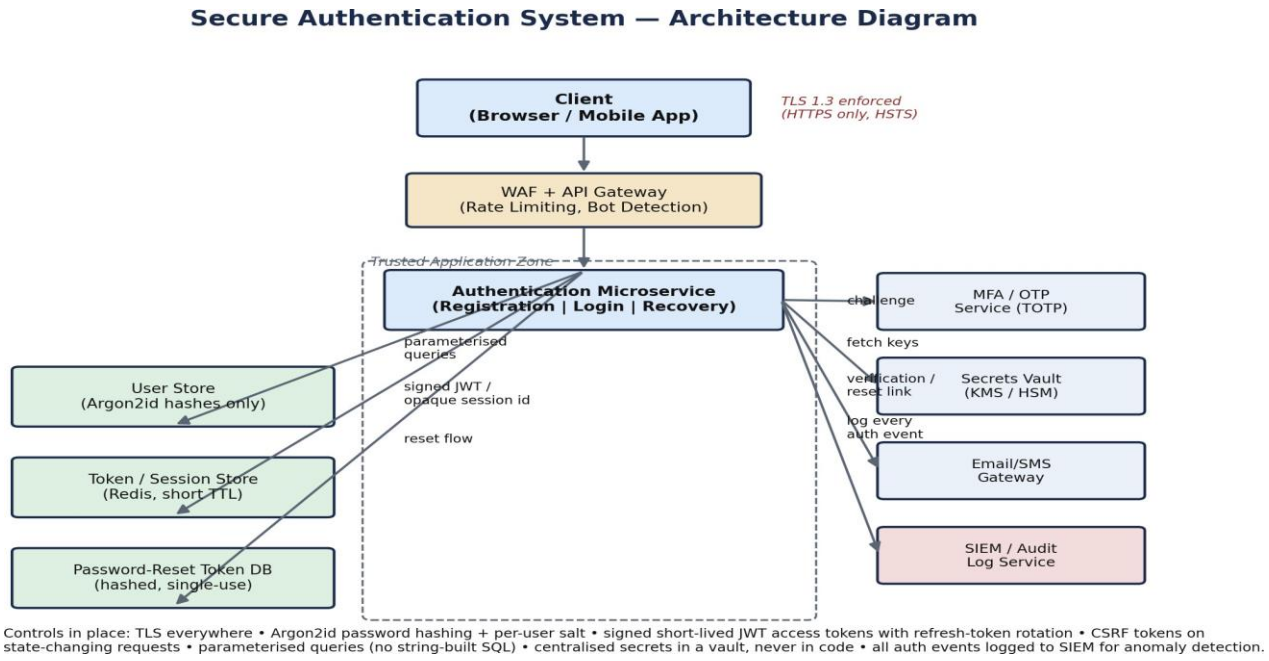

Figure 1. High-level architecture of the secure authentication system, showing trust boundaries and the security control at each hop.

3. Secure Workflow Mapping

3.1 Registration Workflow

Registration validates and sanitises all input on the server, checks email uniqueness with a response that is identical whether or not the account exists (to prevent enumeration), enforces a password policy including a breach-list check, and hashes the password with Argon2id and a per-user salt before persisting the record in an ‘unverified’ state. A signed, short-lived verification token activates the account once confirmed by email.

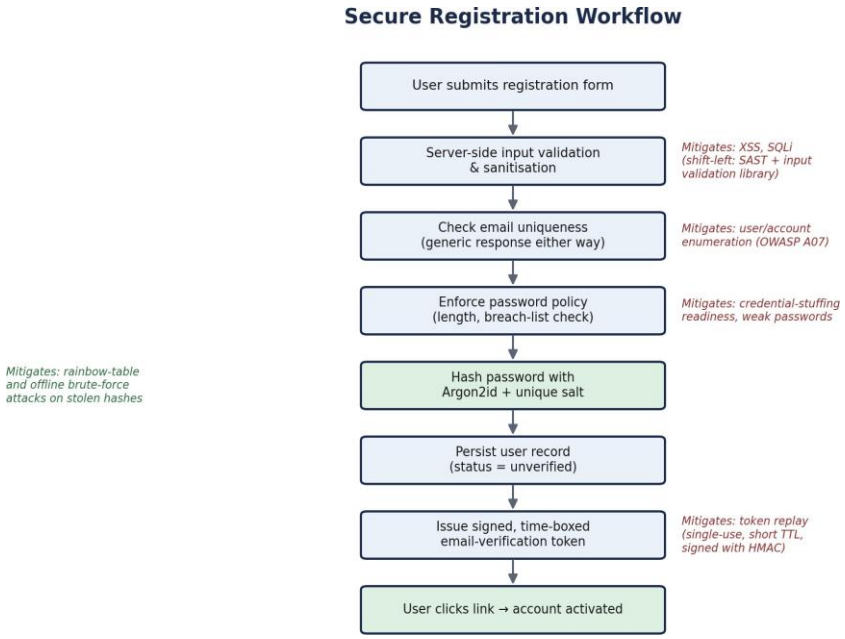

Figure 2. Registration workflow with the specific threat each control mitigates.

3.2 Login Workflow

Login begins with a per-account and per-IP rate/lockout check to blunt brute-force and credential-stuffing attempts, followed by a constant-time comparison against the stored Argon2id hash to avoid timing side-channels. A successful password check triggers an MFA challenge before a short-lived JWT access token and a rotating refresh token are issued and bound to a device fingerprint. Every login event, successful or failed, is logged to the SIEM.

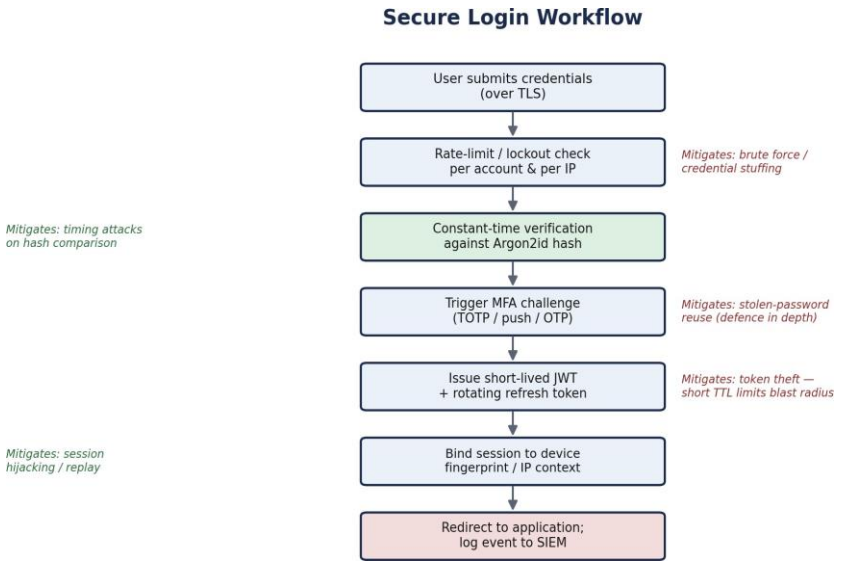

Figure 3. Login workflow with defence-in-depth controls at each stage.

3.3 Password Recovery Workflow

Recovery is designed so that a compromised reset token cannot be reused or leaked from the database: the server returns a generic response regardless of whether the email exists, generates a 256-bit random token, and stores only its hash with a 15-minute expiry. On successful validation the user sets a new password, which is re-hashed, and the reset invalidates the token together with every other active session for that user, closing the window for an attacker holding a hijacked session.

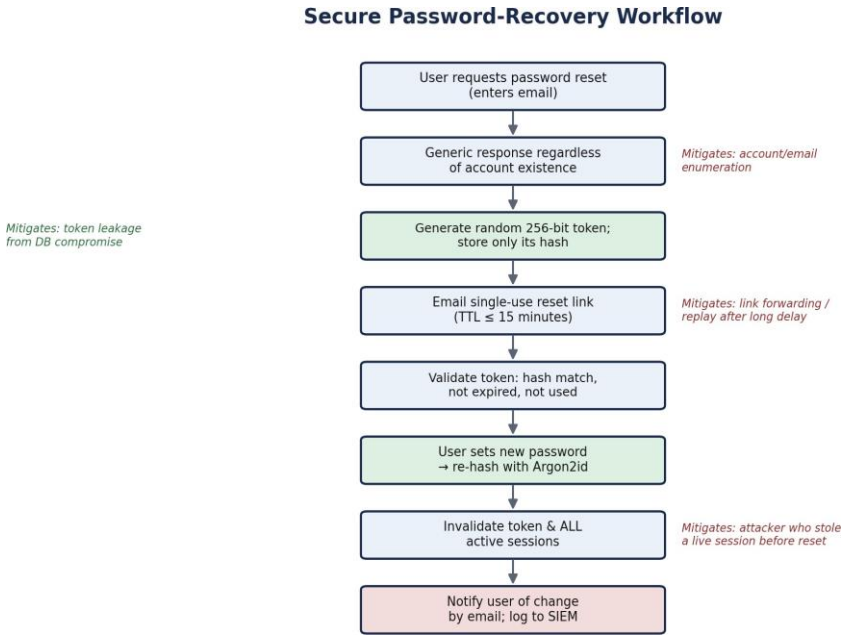

Figure 4. Password-recovery workflow, single-use token life cycle.

4. Threat Modeling (STRIDE)

Each workflow component is assessed against the six STRIDE threat categories — Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege — with the corresponding mitigation already built into the design.

STRIDE Category	Representative Threat	Applied Mitigation
Spoofing	Attacker impersonates a legitimate user during login	MFA, device-bound sessions, signed JWTs with short TTL
Tampering	Modification of reset tokens or session cookies in transit	TLS 1.3, HMAC-signed tokens, HttpOnly + Secure + SameSite cookies
Repudiation	User denies performing a password change or login	Immutable audit log in SIEM with timestamp, IP, and event ID for every auth action
Information Disclosure	Leakage of password hashes, tokens, or account existence	Argon2id hashing, hashed reset tokens, generic responses to prevent enumeration
Denial of Service	Credential-stuffing or scripted flood of the login/registration endpoint	WAF rate limiting, CAPTCHA on repeated failures, exponential back-off lockout
Elevation of Privilege	Broken access control lets a user escalate role after login	Least-privilege JWT claims, server-side authorisation checks on every request

4.1 Component-Level Threat Summary

Component	Key Threat(s)	Mitigation
Client / Browser	Session hijacking, XSS-based token theft	HttpOnly cookies, CSP headers, output encoding
WAF / API Gateway	Bypass via header spoofing, DoS flood	Strict schema validation, layered rate limiting
Auth Microservice	SQL injection, business-logic abuse (e.g. race conditions on reset)	Parameterised queries, idempotent token consumption with row-level locking
User / Token Stores	Data-at-rest exposure if database is breached	Argon2id hashes only, encryption at rest, least-privilege DB accounts
Email/SMS Gateway	Interception or spoofing of verification/reset messages	SPF/DKIM/DMARC, short-lived single-use links

5. Shift-Left Security Mapping (Apply – L3)

Shift-left security moves security activities earlier in the SDLC so that vulnerabilities are caught during design and development rather than after deployment. The table below applies this principle to the registration, login, and recovery workflows designed in this assessment, mapping each SDLC phase to a concrete DevSecOps activity and tool category.

SDLC Phase	Applied Security Activity	Representative Tooling
Requirements	Define abuse-case stories (e.g. 'attacker enumerates emails via registration'); set security acceptance criteria	Threat-modeling workshop, security user stories
Design	Produce the architecture and STRIDE threat model in Sections 2 and 4; choose Argon2id, JWT rotation, MFA up front	STRIDE / DFDs, architecture review
Development	Use vetted auth libraries instead of hand-rolled crypto; enforce input validation and parameterised queries in code	SAST, pre-commit secret scanning, linters
Build / CI	Fail the pipeline on critical SAST findings or known-vulnerable dependencies before merge	SCA (dependency scanning), SAST gates in CI
Testing	Automated tests for rate-limit bypass, token replay, and password-policy edge cases	DAST, fuzzing of auth endpoints, unit tests for token TTL logic
Release / Deploy	Verify TLS configuration, secrets are pulled from vault (not config files), and WAF rules are active	IaC scanning, container image scanning
Operate / Monitor	Continuous monitoring of auth events in SIEM; alert on anomalous login patterns	SIEM correlation rules, runtime application self-protection

By pushing controls such as SAST, dependency scanning, and threat modeling into the requirements, design, and CI stages, most of the defects identified in Section 4 are caught and fixed before the authentication service ever reaches production — the practical demonstration of shift-left security for this system.

6. Conclusion

The proposed design applies well-established security principles — strong password hashing, short-lived and rotating tokens, multi-factor authentication, rate limiting, and centralised audit logging — to all three phases of the authentication lifecycle. STRIDE threat modeling confirms that each major threat category has a corresponding mitigation in the architecture, and the shift-left mapping shows how these controls are embedded from the requirements phase onward rather than bolted on after release, satisfying the Apply (L3) objective of translating security

theory into a concrete, defensible system design.